

Should we build a prompt-helper for our employees?

A build/don't-build recommendation

Question: staff write weak prompts to AI chatbots.

Do we build a tool that helps them write better ones?

Evidence base: hands-on evaluation of Prompt Butter Jam (promptbutterjam.com) — the closest existing implementation of this exact idea — tested 2026-08-02.

Bottom line up front: don't build the form. Build the two things underneath it.

The idea has a real kernel

Structured prompts *do* outperform free-form ones. That part is not in doubt.

But of the seven fields a tool like this typically collects, only about three are load-bearing:

Field	Verdict
Stop condition	✅ Strongest lever. Models over-deliver by default
Output shape	✅ Real effect — "diagnosis then patch" vs "patch only"
Context / the actual code	✅ Decisive — but this is just "include the code"
Role ("act as a senior engineer")	⚠️ Weak, inconsistent on modern models
Task type · Complexity · Detail	⚠️ Mostly restates the goal sentence

Design implication: a seven-field form spends most of its UI budget on the fields that don't matter.

The fatal flaw: it optimizes the wrong half

A prompt-composer can only rearrange **what the user already typed**.

Observed directly in testing — the tool emitted:

```
SCOPE:  
- Stack: Ruby on Rails, PostgreSQL  
CONTEXT:  
def charge_order(order_id, token): ← Python
```

It had no idea what the user was working on. It labelled Python code as Rails because "Rails" was a leftover default in a field.

The difference between a prompt that works and one that doesn't is almost always *which context was included* — not whether the sections had headers.

A form polishes the wrapper. It cannot touch the contents.

What that means for a build decision

The vendor's own roadmap confirms it: their headline waitlist feature is "**codebase-aware sharpening.**"

They know the form isn't the product. The context is.

So a build team faces a fork:

- Build the **form** → ships fast, demos well, moves the metric barely
- Build the **context layer** → the real problem, and genuinely hard

Building the form because it's the tractable half is the classic streetlight error. It is the part that was never broken.

Second flaw: friction lands in the wrong place ⚠️

A separate destination — a web form you visit *before* doing work — is a tax paid manually, on every prompt, forever.

- User is already in a chat window
- Alt-tab → fill 7 fields → copy block back → paste
- ~30 seconds, every time, self-administered

Adoption reality: tools that require remembering to use them get used during the pilot and abandoned by week three.

The people who most need prompt discipline are exactly the people least likely to detour through a form to get it.

Any build must survive this test: **does it work without the user choosing to invoke it?**

Third flaw: it creates a data-governance liability ●

Tested by pasting credentials into the context box:

```
AWS_SECRET_ACCESS_KEY=wJa1rXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY  
DB_PASSWORD=hunter2  
ghp_16CharsRealLookingToken000000000
```

All three passed through verbatim into the output prompt —
while that same prompt asserted `EXCLUDE: Secrets, customer data...`

That line is an *instruction to the assistant*, not enforcement.
It reads as a safety guarantee and is not one.

If you build this, a text box labelled "paste your context here" becomes a funnel that actively invites staff to paste secrets.
Redaction is then *your* requirement, not a nice-to-have.

Alternatives, ranked by value per unit of effort

#	Approach	Effort	Value	Fires automatically?
1	System-prompt / config defaults	Very low	High	✓ Yes
2	Assistant asks when info is missing	Low	High	✓ Yes
3	Templates in the tools staff already use	Low	Medium	⚠ If discoverable
4	Training + a one-page checklist	Low	Medium	✗ No
5	Context/retrieval layer	High	Highest	✓ Yes
6	A prompt-composer form (PBJ-style)	Medium	Low	✗ No

The thing being proposed ranks last. Every cheaper option beats it, and the one expensive option that beats it is a different product entirely.

Alternative 1 — put the discipline in the system prompt

If you run any assistant with a configurable system prompt (enterprise ChatGPT, Copilot, a Claude deployment, an internal wrapper):

Encode the checklist once. It applies to every prompt, from every employee, automatically.

- Captures the two fields that actually matter (stop condition, output shape)
- **Zero user friction** — nobody has to remember anything
- One config change, not a product
- Updates centrally when you learn something new

This is strictly better than a form at the form's own job,
because the discipline gets applied without anyone choosing to apply it.

Start here. It is days of work, not quarters.

Alternative 2 — invert it: make the assistant ask

Instead of a form the user fills in *advance*:

"If the request is ambiguous or has no clear stopping point, ask one clarifying question before starting."

Why this beats a form:

- Fires **only when something is actually missing** — no tax on the ~70% of prompts that were already fine
- The user answers in the chat they're already in — no context switch
- It teaches by example, repeatedly, in context

PBJ's static "QUERY TO THE AUTHOR" panel is a clumsy imitation of this. A conversation does it natively and better.

Alternative 5 — the one worth real engineering

If you want to spend a team on this, spend it on **context, not phrasing**.

The unsolved problem is: *which* 50K tokens should go in the prompt?

- Wire the assistant to the systems staff actually reference — docs, tickets, code, runbooks
- Retrieval means users stop hand-pasting context, which is both the quality bottleneck **and** the secrets-leak vector

This is the expensive option, and the only one that addresses the flaw on slide 3. It is also a well-understood problem with mature tooling — you would not be inventing a category.

Do not build a form as a cheap substitute for this.
They solve different problems.

Recommendation

Don't build the prompt-composer form. It optimizes the half of the problem that wasn't broken, requires users to remember to use it, and creates a secrets-collection surface you'd then have to secure.

Do, in this order:

1. **Ship the defaults in the system prompt** — days of work, applies to everyone
2. **Add the clarifying-question behavior** — catches what defaults can't
3. **Publish a one-page checklist** — GOAL / CONTEXT / CONSTRAINTS / OUTPUT / STOP WHEN
4. **Then evaluate a retrieval/context layer** — the real problem, real effort

Revisit the build question only if you find that (1)–(3) shipped and measurably failed. Then you'd know the gap is user *behavior*, not defaults — which is the only case a form actually addresses.

Caveats on this evidence

Prior art, not a competitor. PBJ is an independent pre-release side project by one developer, tested **2026-08-02**. Its bugs are not the argument — a well-funded team would fix them. The argument is structural: the *category* has the flaws on slides 3–6.

Its AI "sharpen" pass is not live — currently a hardcoded demo. If a real critique pass ships (a second model flagging that "make it fast" isn't checkable), that is a genuine capability that defaults alone don't replicate. **That would be worth re-testing**, and it's the one scenario that could change this recommendation.

Credit where due: PBJ's privacy claim was verified true — prompts genuinely never leave the browser. The secrets finding is about what the tool *invites users to type*, not about exfiltration.